INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Machine Learning

Rosas Carrillo Ary Shared

Práctica 2: Clasificador K-NN y K-NN Ponderado

Herrera Mauricio Pedro Alonso – 2020600448

Corro Mendoza Onasis Alejandro – 2022630202

Ciudad de México

24 de March de 2026

Índice

1. Introducción . . . 3
2. Marco teórico . . . 3
   2.1. El algoritmo K-NN . . . 3
   2.2. Distancia euclidiana . . . 3
   2.3. K-NN ponderado por distancia . . . 4
   2.4. Método de evaluación por restitución . . . 4
3. Desarrollo . . . 4
   3.1. Datasets utilizados . . . 4
      3.1.1. Dataset Iris . . . 4
      3.1.2. Dataset Breast Cancer Wisconsin . . . 5
   3.2. Implementación . . . 5
      3.2.1. Carga de datos (`dataset.py`) . . . 5
      3.2.2. K-NN clásico (`knn.py`) . . . 6
      3.2.3. K-NN ponderado (`knn_ponderado.py`) . . . 6
      3.2.4. Programa principal (`main.py`) . . . 6
4. Pruebas y resultados . . . 7
   4.1. K-NN clásico sobre Iris . . . 7
   4.2. K-NN ponderado sobre Breast Cancer . . . 7
   4.3. Comparación entre variantes . . . 7
5. Conclusiones . . . 8
Referencias . . . 8

1. Introducción

El algoritmo K-Nearest Neighbors (K-NN) clasifica un punto nuevo buscando los k puntos más cercanos en el conjunto de entrenamiento y asignándole la clase que más se repite entre ellos [1]. No construye un modelo como tal: solo guarda los datos y hace el cálculo al momento de predecir.

En esta práctica se implementaron dos variantes en Python:

- **K-NN clásico**: cada vecino tiene un voto igual, gana la clase con más votos.
- **K-NN ponderado**: los vecinos más cercanos pesan más en la votación, según la fórmula de Dudani [2].

Se probaron ambas variantes con evaluación por restitución sobre el dataset Iris [3] y el dataset Breast Cancer Wisconsin [4].

2. Marco teórico

2.1. El algoritmo K-NN

Fix y Hodges propusieron el K-NN en 1951 [5], y Cover y Hart lo formalizaron en 1967 [1]. Es un método de aprendizaje basado en instancias (*instance-based learning*): no ajusta parámetros en una fase de entrenamiento, sino que guarda todos los ejemplos y calcula las distancias cuando llega un punto nuevo a clasificar [6].

Dado un conjunto de entrenamiento $\mathcal{D} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), ..., (\boldsymbol{x}_n, y_n)\}$, donde $\boldsymbol{x}_i \in \mathbb{R}^d$ es un vector de características y y_i es su etiqueta de clase, el algoritmo clasifica un nuevo punto $\boldsymbol{x}_q$ de la siguiente manera:

1. Calcular la distancia $d(\boldsymbol{x}_q, \boldsymbol{x}_i)$ entre $\boldsymbol{x}_q$ y cada punto $\boldsymbol{x}_i$ del conjunto de entrenamiento.
2. Seleccionar los k puntos con menor distancia: el conjunto $\mathcal{N}_k(\boldsymbol{x}_q)$.
3. Asignar a $\boldsymbol{x}_q$ la clase más frecuente entre los k vecinos.

La regla de decisión se expresa formalmente como:

$$\hat{y} = \arg\max_{c \in \mathcal{C}} \sum_{\boldsymbol{x}_i \in \mathcal{N}_k(\boldsymbol{x}_q)} \mathbb{1}(y_i = c)$$

donde $\mathbb{1}(\cdot)$ es la función indicadora y $\mathcal{C}$ es el conjunto de clases posibles [7].

2.2. Distancia euclidiana

La métrica más común para K-NN es la distancia euclidiana. Para dos puntos $\boldsymbol{x}_a = (x_{a1}, x_{a2}, ..., x_{ad})$ y $\boldsymbol{x}_b = (x_{b1}, x_{b2}, ..., x_{bd})$ en $\mathbb{R}^d$:

$$d(\boldsymbol{x}_a, \boldsymbol{x}_b) = \sqrt{\sum_{j=1}^{d} \left(x_{aj} - x_{bj}\right)^2}$$

Hay que tener en cuenta que esta métrica trata todas las características por igual; si un atributo tiene un rango mucho mayor que otro, domina el cálculo de distancia [8].

2.3. K-NN ponderado por distancia

En el K-NN clásico, un vecino que apenas entra entre los k más cercanos cuenta igual que el vecino más próximo. Eso puede ser problemático cuando k es grande. Dudani [2] propuso asignar un peso W_i a cada vecino para que los más cercanos cuenten más:

La fórmula de ponderación propuesta por Dudani es:

$$W_i = \begin{cases} \dfrac{d_k - d_i}{d_k - d_1} & \text{si } d_k \neq d_1 \\ 1 & \text{si } d_k = d_1 \end{cases}$$

donde:

- d_i es la distancia del i-ésimo vecino al punto de consulta,
- d_1 es la distancia al vecino más cercano (el primero),
- d_k es la distancia al vecino más lejano (el k-ésimo).

El vecino más cercano obtiene $W_1 = 1$ y el más lejano $W_k = 0$. Los demás quedan entre esos dos valores, proporcionales a qué tan cerca están.

La regla de decisión ponderada queda:

$$\hat{y} = \arg\max_{c \in \mathcal{C}} \sum_{\boldsymbol{x}_i \in \mathcal{N}_k(\boldsymbol{x}_q)} W_i \cdot \mathbb{1}(y_i = c)$$

Cuando $d_k = d_1$ (todos los vecinos están a la misma distancia), la fórmula daría $\frac{0}{0}$, así que se asigna peso uniforme $W_i = 1$ [2].

2.4. Método de evaluación por restitución

La restitución (*resubstitution*) consiste en evaluar el clasificador con los mismos datos que se usaron para entrenarlo. Se clasifica cada punto y se compara contra su etiqueta real [7]:

$$\text{Precisión} = \frac{\text{Número de clasificaciones correctas}}{\text{Número total de muestras}} \times 100\%$$

Este método da una estimación optimista porque el clasificador ya "conoce" los datos. No mide qué tan bien generaliza a datos nuevos. Aun así, sirve para verificar que la implementación funciona y para comparar el efecto de distintos valores de k.

3. Desarrollo

3.1. Datasets utilizados

3.1.1. Dataset Iris

Fisher publicó el dataset Iris en 1936 [3]. Tiene 150 muestras de tres especies de flores: *Iris-setosa*, *Iris-versicolor* e *Iris-virginica* (50 de cada una). Cada muestra se describe con 4 mediciones:

Característica	Unidad	Descripción
SepalLengthCm	cm	Largo del sépalo
SepalWidthCm	cm	Ancho del sépalo
PetalLengthCm	cm	Largo del pétalo
PetalWidthCm	cm	Ancho del pétalo

Tabla 1: Características del dataset Iris.

Este dataset se utilizó para evaluar el K-NN clásico con voto mayoritario.

3.1.2. Dataset Breast Cancer Wisconsin

El dataset Breast Cancer Wisconsin [4] tiene 568 biopsias de tumores mamarios etiquetadas como malignas (M) o benignas (B). De cada imagen digitalizada del aspirado con aguja fina se extraen 10 mediciones (radio, textura, perímetro, área, suavidad, compacidad, concavidad, puntos cóncavos, simetría y dimensión fractal), y de cada una se reporta la media, el error estándar y el peor valor, lo que da 30 características por muestra.

Dataset	Muestras	Características
Iris	150	4
Breast Cancer	568	30

Tabla 2: Resumen de los datasets utilizados.

Este dataset se utilizó para evaluar el K-NN ponderado.

3.2. Implementación

El código se organizó en cuatro archivos de Python:

3.2.1. Carga de datos (`dataset.py`)

La clase `Dataset` lee un CSV con Polars, descarta columnas que no aportan (como el ID) y separa las características numéricas de la columna de clase:

```python
class Dataset:
    def __init__(self, ruta_csv, columna_clase, columnas_excluir=None):
        df = pl.read_csv(ruta_csv)
        df = df.drop(col for col in df.columns if df[col].is_null().all())
        if columnas_excluir:
            df = df.drop(columnas_excluir)
        self.clases = df[columna_clase].cast(pl.Utf8).to_list()
        self.nombres_clases = sorted(set(self.clases))
        df_caracteristicas = df.drop(columna_clase)
        self.caracteristicas = [
            [float(valor) for valor in fila]
            for fila in df_caracteristicas.rows()
        ]
```

3.2.2. K-NN clásico (`knn.py`)

La clase `KNN` guarda los datos en `entrenar` (no hay ajuste de parámetros) y hace todo el trabajo en `clasificar`: calcula la distancia a cada punto, ordena, toma los k más cercanos y retorna la clase más frecuente con `Counter`:

```python
def clasificar(self, punto_nuevo):
    distancias = []
    for i in range(len(self.datos_entrenamiento)):
        dist = self._distancia_euclidiana(punto_nuevo,
                                          self.datos_entrenamiento[i])
        distancias.append((dist, self.clases_entrenamiento[i]))
    distancias.sort(key=lambda x: x[0])
    k_vecinos = [clase for _, clase in distancias[:self.k]]
    conteo = Counter(k_vecinos)
    return conteo.most_common(1)[0][0]
```

3.2.3. K-NN ponderado (`knn_ponderado.py`)

`KNNPonderado` hereda de `KNN` y solo redefine `clasificar`. En lugar de contar votos iguales, acumula el peso $W_i = \frac{d_k - d_i}{d_k - d_1}$ por clase y elige la de mayor peso total:

```python
def clasificar(self, punto_nuevo):
    # ... calcular y ordenar distancias ...
    k_vecinos = distancias[:self.k]
    for dist, clase in k_vecinos:
        if dist == 0.0:
            return clase  # punto idéntico, evita div/0
    d1 = k_vecinos[0][0]   # distancia mínima
    dk = k_vecinos[-1][0]  # distancia máxima
    pesos_por_clase = {}
    for dist, clase in k_vecinos:
        if dk == d1:
            peso = 1.0     # equidistantes: peso uniforme
        else:
            peso = (dk - dist) / (dk - d1)
        if clase in pesos_por_clase:
            pesos_por_clase[clase] += peso
        else:
            pesos_por_clase[clase] = peso
    return max(pesos_por_clase, key=lambda c: pesos_por_clase[c])
```

Hay dos guardas contra la división entre cero: si un vecino tiene distancia 0, se regresa su clase de inmediato; si $d_k = d_1$, se usa peso 1 para todos.

3.2.4. Programa principal (`main.py`)

Desde la línea de comandos se elige qué variante correr y con qué valores de k:

```python
# Uso:
# python main.py clasico 1,3,5,7
# python main.py ponderado 1,3,5,7
```

El modo `clasico` carga Iris y usa `KNN`; el modo `ponderado` carga Breast Cancer y usa `KNNPonderado`. En ambos casos se evalúa por restitución con cada k.

4. Pruebas y resultados

4.1. K-NN clásico sobre Iris

Resultados de la evaluación por restitución con voto mayoritario:

k	Precisión (%)
1	100.00
3	$\approx$ 96.00
5	$\approx$ 96.67
7	$\approx$ 96.67

Tabla 3: Resultados de K-NN clásico sobre Iris (evaluación por restitución).

Con $k = 1$ se obtiene 100% porque cada punto es su propio vecino más cercano (distancia 0). Al subir k, la precisión baja un poco: algunos puntos en la frontera entre *Iris-versicolor* e *Iris-virginica* quedan mal clasificados cuando entran vecinos de la otra clase.

4.2. K-NN ponderado sobre Breast Cancer

Resultados con la ponderación de Dudani:

k	Precisión (%)
1	100.00
3	$\approx$ 97.00
5	$\approx$ 97.00
7	$\approx$ 97.00

Tabla 4: Resultados de K-NN ponderado sobre Breast Cancer (evaluación por restitución).

Aquí la precisión se mantiene estable al subir k. Tiene sentido: el vecino más lejano de los k recibe peso cercano a 0, así que aunque sea de otra clase, no alcanza a cambiar el resultado.

4.3. Comparación entre variantes

En el K-NN clásico, pasar de $k = 1$ a $k = 7$ implica que vecinos más distantes votan con el mismo peso que el más cercano. Eso puede confundir la clasificación en zonas donde las clases están cerca unas de otras. Con la ponderación de Dudani este problema se reduce, porque el peso del vecino más lejano tiende a cero.

5. Conclusiones

El K-NN clásico funcionó bien sobre Iris, con precisiones arriba del 96% para $k > 1$. La versión ponderada de Dudani se comportó mejor conforme k crece, porque el peso del vecino más lejano tiende a cero y no "contamina" la votación.

La evaluación por restitución es optimista por definición (el clasificador ya vio los datos), pero cumplió su propósito: verificar que el código funciona y observar cómo cambia la precisión con distintos k.

En cuanto a la implementación, hacer que `KNNPonderado` herede de `KNN` permitió reutilizar la distancia euclidiana y el ciclo de evaluación, cambiando solo la lógica de votación.

Quedan cosas por explorar. La evaluación por restitución no dice nada sobre generalización; valdría la pena usar validación cruzada. También habría que normalizar las características antes de calcular distancias, sobre todo con el dataset de Breast Cancer, donde los rangos de las 30 variables son muy distintos entre sí.

Referencias

[1] T. Cover y P. Hart, "Nearest neighbor pattern classification", *IEEE Transactions on Information Theory*, vol. 13, núm. 1, pp. 21–27, 1967.

[2] S. A. Dudani, "The distance-weighted k-nearest-neighbor rule", *IEEE Transactions on Systems, Man, and Cybernetics*, núm. 4, pp. 325–327, 1976.

[3] R. A. Fisher, "The use of multiple measurements in taxonomic problems", *Annals of Eugenics*, vol. 7, núm. 2, pp. 179–188, 1936.

[4] W. H. Wolberg, W. N. Street, y O. L. Mangasarian, "Breast Cancer Wisconsin (Diagnostic) Data Set". 1995.

[5] E. Fix y J. L. Hodges, "Discriminatory analysis. Nonparametric discrimination: Consistency properties", *International Statistical Review*, vol. 57, núm. 3, pp. 238–247, 1951.

[6] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.

[7] R. O. Duda, P. E. Hart, y D. G. Stork, *Pattern Classification*, 2a ed. Wiley-Interscience, 2001.

[8] T. Hastie, R. Tibshirani, y J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2a ed. Springer, 2009.